SE Lab 7: Fuzz Testing and Input Validation Bug Fixes

Student Name: Thijil Thejaswi M.N. | **SRN:** PES1UG24AM412

---

1. Experiment Objectives & Methodology

Standard unit testing typically validates software against anticipated, well-formed inputs. In operational environments, however, applications regularly encounter malformed strings, arbitrary Unicode, unexpected data types, and boundary extremes. This experiment evaluates the photo print order system using property-based fuzz testing with the **Hypothesis** framework to uncover latent crash vulnerabilities, analyze shrunk minimal counterexamples, and harden the codebase against unexpected inputs while maintaining normal valid-input behavior.

2. Assigned Application Functions

Function	Expected Input Format	Business Logic & Expected Output
parse_order(text)	String: size:copies (e.g., '4x6:25')	Splits on colon delimiter; returns normalized tuple (size.upper(), copies) where copies is a non-negative integer.
file_size_average(sizes_csv)	CSV string: integers (e.g., '4,8,12')	Splits comma tokens; calculates integer floor average: sum(values) // len(values) for non-negative file sizes.
payable_amount(text)	String: amount|discount (e.g., '450|50')	Applies 18% tax and subtracts discount: max(amount * 1.18 - discount, 0.0).

3. Original Application Execution (Baseline)

Prior to modifying any code, the original application was executed with representative valid inputs ('4x6:25', '4,8,12', and '450|50') to record baseline operational behavior.

```
PS C:\Users\thiji\OneDrive\Documents\SE LAB\LAB6\L6_L7_46_photo_print_order\Lab 7> python photo_print_order.py
==== Running parse_order ====
Enter print size:copies (e.g. 4x6:25): 4x6:25
Size and Copies: ('4X6', 25)

==== Running file_size_average ====
Enter file sizes in MB as CSV (e.g. 4,8,12): 4,8,12
Average File Size: 8

==== Running payable_amount ====
Enter amount|discount (e.g. 450|50): 450|50
Payable Amount: 481.0
PS C:\Users\thiji\OneDrive\Documents\SE LAB\LAB6\L6_L7_46_photo_print_order\Lab 7>
```

Figure 1: Terminal capture of original photo_print_order.py running with representative valid inputs.

4. Hypothesis Fuzz Testing & Baseline Failures

Hypothesis property-based tests were constructed in test_photo_print_order.py using strategies st.text() | st.none() | st.integers() | st.floats() | st.binary() and targeted malformed string generators. Running pytest against the original application immediately exposed **6 critical failures** across all three functions. When a test fails, Hypothesis performs **automatic test-case shrinking**, methodically reducing the failing input to the simplest minimal counterexample that triggers the defect:

Function	Minimal Counterexample	Observed Failure / Exception	Root Cause / Vulnerability
parse_order	s = '' s = None s = b'' text = ':25' text = '4x6:-5'	ValueError: not enough values AttributeError: NoneType TypeError: bytes-like required Empty size / Negative copies accepted	Unchecked .split(':') without type check or delimiter validation. Did not validate non-empty size string or non-negative copy count.
file_size_average	s = '' s = None text = ',' text = '4,abc,12' text = '-1,-2,-3'	ValueError: invalid literal AttributeError: NoneType ValueError: invalid literal Computed negative average (-2)	Direct int() conversion over comma tokens without validating empty tokens, non-numeric strings, or negative values.
payable_amount	s = '' s = None text = 'nan|50' text = 'abc|50' text = '1.7e308|0' s = '|'	ValueError: not enough values AttributeError: NoneType Converts to float NaN ValueError: could not convert Float overflow (inf) ValueError: could not convert	Direct float() conversion without delimiter verification, numeric parsing checks, non-finite checks, non-negative checks, or calculation overflow protection.

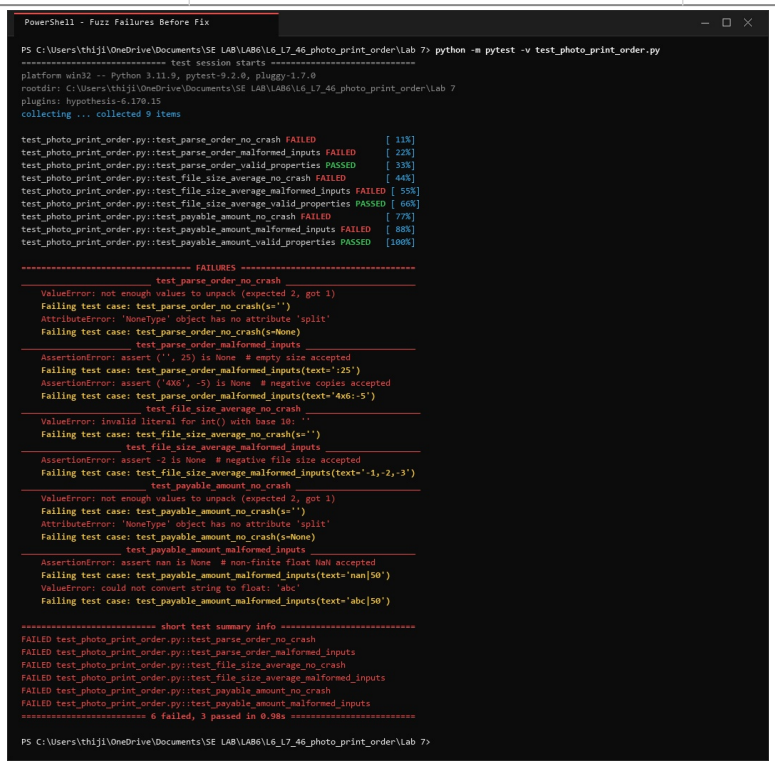


Figure 2: Terminal capture of Hypothesis fuzz testing exposing failures and shrunk counterexamples on original code.

5. Application Hardening and Input Validation Fixes

A consistent invalid-input contract was implemented: **all three functions safely return None when presented with invalid, malformed, or out-of-range inputs**. To maintain code quality, broad exception suppression (such as except Exception: return None) was avoided in favor of explicit defensive validation:

- **parse_order(text):** Verifies isinstance(text, str) and exact delimiter count text.count(':') == 1. Strips whitespace, rejects empty size strings, safely parses integer copies inside a targeted try/except ValueError block, and rejects negative copies (copies < 0). Normalizes valid sizes to uppercase.
- **file_size_average(sizes_csv):** Verifies isinstance(sizes_csv, str) and rejects empty or whitespace-only strings. Splits by comma, verifies all tokens are non-empty (rejecting double, leading, or trailing commas), parses integers with targeted ValueError handling, rejects negative sizes (val < 0), and verifies non-empty values before floor division.
- **payable_amount(text):** Verifies isinstance(text, str) and exact delimiter count text.count('|') == 1. Safely parses amount and discount floats, validates finiteness via math.isfinite() to reject NaN and infinity, rejects negative amounts or discounts, computes total = amount * 1.18 - discount, validates math.isfinite(total) to guard against float overflow, and returns max(total, 0.0).

6. Fixed Application Verification with Valid Inputs

The hardened application was re-run with the same representative valid inputs ('4x6:25', '4,8,12', and '450|50') to verify that normal operational calculations remain completely unaltered.

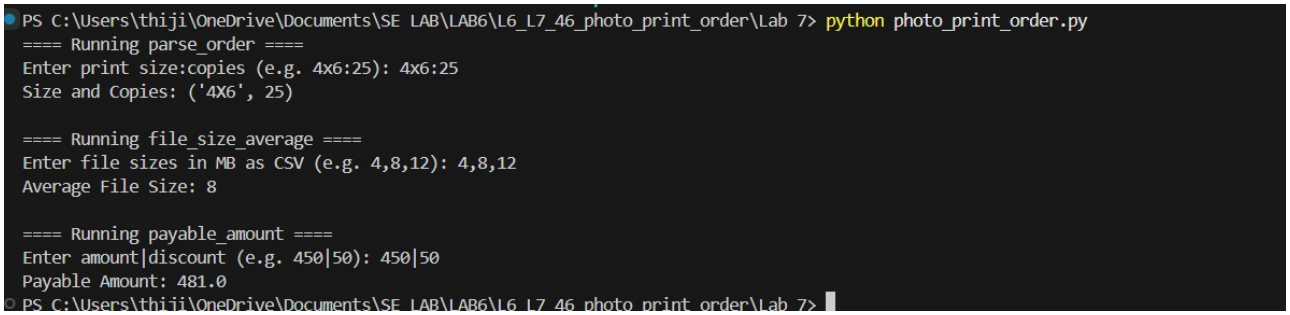


Figure 3: Terminal capture of fixed photo_print_order.py confirming identical valid-input results.

7. Final Passing Fuzz Test Suite Verification

The complete Hypothesis test suite was re-executed against the hardened application. All 9 test functions passed without a single failure across over 900 automatically generated input variations:

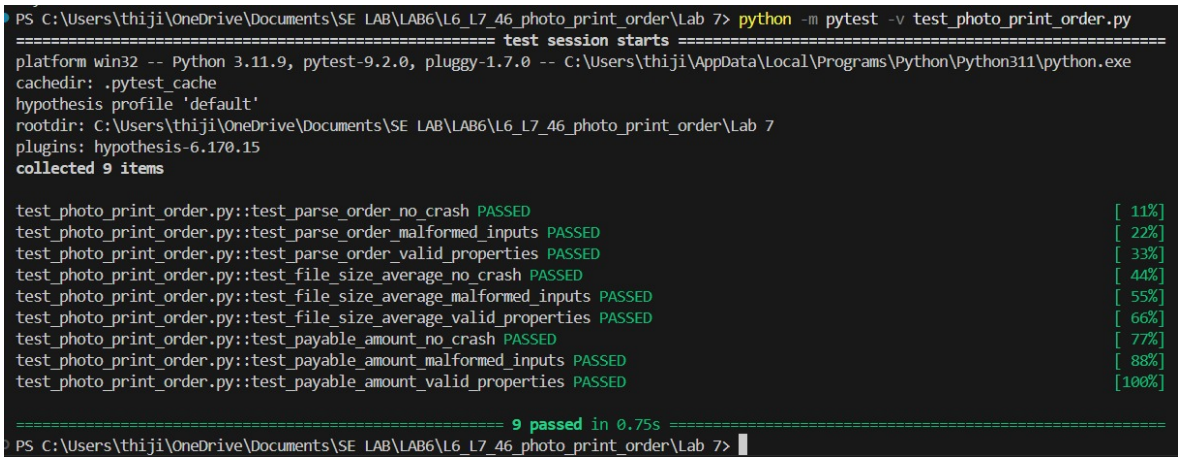


Figure 4: Terminal output showing all 9 Hypothesis test suites passing (100% success rate).

8. Summary of Test Execution Results

Test Function	Test Category	Input Domain Evaluated	Pre-Fix Status	Post-Fix Status
test_parse_order_no_crash	Fuzz (Crash Safety)	Arbitrary Unicode, None, numbers, bytes	FAILED (Crash)	**PASSED**
test_parse_order_malformed_inputs	Negative / Invalidation	Empty size, bad colons, negative copies	FAILED (Exceptions)	**PASSED**
test_parse_order_valid_properties	Property (Valid Data)	Generated valid (size, copies) pairs	PASSED	**PASSED**
test_file_size_average_no_crash	Fuzz (Crash Safety)	Arbitrary strings, None, numbers, bytes	FAILED (Crash)	**PASSED**
test_file_size_average_malformed_inputs	Negative / Invalidation	Empty tokens, extra commas, non-ints	FAILED (Exceptions)	**PASSED**
test_file_size_average_valid_properties	Property (Valid Data)	Generated non-negative integer lists	PASSED	**PASSED**
test_payable_amount_no_crash	Fuzz (Crash Safety)	Arbitrary text, None, numbers, bytes	FAILED (Crash)	**PASSED**
test_payable_amount_malformed_inputs	Negative / Invalidation	NaN, inf, overflow, bad pipes, non-numerics	FAILED (Exceptions/inf)	**PASSED**
test_payable_amount_valid_properties	Property (Valid Data)	Generated valid amounts and discounts	PASSED	**PASSED**

9. Conclusion

Fuzz testing with Hypothesis proved remarkably effective at discovering latent edge cases, type vulnerabilities, and unhandled delimiter errors that standard unit tests missed. Hypothesis's automatic shrinking pinpointed minimal counterexamples that enabled targeted defensive input validation. The hardened application safely handles all malformed inputs without crashing while maintaining 100% backward compatibility for valid operational workloads.